

DeepSeek Harness 新手教程白皮书

从零上手 DeepSeek Harness (dsh) ——DeepSeek 官方开源的 Agent 运行时与插件生态。本白皮书面向第一次接触 dsh 的开发者，用真实可跑的示例带你走完：安装 → 使用 → 开发插件 → 实战调优。

[English](#) (规划中) · [GitHub](#)

为什么需要这本白皮书

2026-08-13, DeepSeek 正式开源 **DeepSeek Harness (dsh)** ——一个"everything is a plugin"的 Agent 运行时（基于 Cordis）。它的定位介于 Claude Code 与自建 Agent 框架之间：官方提供 Web UI 与 headless CLI，同时开放了完整的插件体系，让社区开发自己的工具、界面与 workflow。

但官方文档以架构说明为主，缺少一条面向新手的上手路径。这本白皮书补上这条路——每章都有可复制、可运行的示例，所有命令均在本机（Windows/macOS/Linux）实体验证。

目录

章节	内容	状态
第 1 章: 认识 DeepSeek Harness	它是什么、架构、与 Claude Code / OpenCode 的差异、插件生态定位	✓
第 2 章: 五分钟快速上手	安装、web / headless 双模式、第一个对话、模型与推理档位	✓
第 3 章: profile 与插件系统	profile 结构、cordis.patch.yml、插件注册、host/client 双半、扩展点、常见坑	✓
第 4 章: 插件开发实战	从零写第一个插件：纯函数决策 + agent/request waterfall + 测试与实机验证	✓
第 5 章: 实战案例	Git 面板、HTML 草稿预览、tool 加速插件——真实代码讲原理	✓
第 6 章: 进阶与性能调优	reasoning_effort 策略、工具耗时分析、真实踩坑清单、评测视角	✓
第 7 章: 生态与资源	官方入口、插件生态、参与路径、阅读建议	✓

白皮书 PDF

- [DeepSeek-Harness-白皮书.pdf](#) (随章节更新重新生成)

第 1 章：认识 DeepSeek Harness

本章目标：回答三个问题——它是什么？它和 Claude Code 有什么区别？我为什么要用它？

1.1 一句话定义

DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 **Agent 运行时**：一个"一切皆插件" (everything is a plugin) 的框架，用于构建、运行和扩展 AI Agent。它基于 Cordis（一个为可组合系统设计的插件容器）构建。

官方仓库的一句话定位：

"DeepSeek Harness (*dsh*) is an open-source agent harness developed by DeepSeek AI. It uses an architecture where **everything is a plugin**."

2026-08-13 开源，MIT 协议，TypeScript 编写。版本线 0.1.0-rc.x （当前 rc.6，迭代极快，官方明示"将有破坏性变更"）。

1.2 它解决什么问题

痛点	dsh 的答案
想要一个能自由定制的 Agent，而不是被闭源工具锁死	官方开源 + MIT + 插件体系，任何行为都可替换
想要"模型 + 工具 + 界面"解耦，各层独立演进	分层：llm（模型）/ tools（工具）/ client（界面）/ session（会话）各自是插件
想要在服务器/CI/终端跑 Agent，而不只是 GUI	官方提供 web（Web UI）与 headless（一次性 CLI）两个 profile
想给社区贡献能力，而不是只能提 issue	CONTRIBUTING 明确鼓励做 dsh-plugin 生态项目

一句话：**dsh 是"你自己 Agent 的乐高底座"**——官方搭好运行时，能力由插件堆出来。

1.3 核心架构：一切皆插件

你的 profile	
(如 dsh web / dsh headless / 你的自定义 profile)	
profile = bundle 栈 + 你的 patch 层	
bundle 例子: @deepseek-ai/dsh-base (核心)	
@deepseek-ai/dsh-web-app (Web UI)	
dsh-better-sidebar (社区插件)	
Cordis 插件容器: 每个能力 = 一个插件	
llm / tools / session / client / settings ...	

关键概念 (第 3 章展开)：

- **profile**: 一个可启动的配置栈 (`$DSH_HOME/profiles/<name>/`)，包含 `package.json` (插件依赖 + 清单) + `cordis.patch.yml` (你的覆盖层)。
- **bundle**: 一组插件的集合 (官方预置 `dsh-base`、`dsh-web-app`、`dsh-headless`)。
- **插件 (plugin)**：cordis 插件，分 **host 半** (Node 侧：工具、服务、事件) 与 **client 半** (浏览器侧：UI、交互) ——一个 npm 包可以同时携带两半。
- **extension point (扩展点)**：官方为"不改核心、只加插件"设计的钩子——如 `agent/request` waterfall、`conversationEvents`、`settings` 命名空间。**新手最容易忽略的是：改行为优先找扩展点，而不是 fork 核心。**

1.4 与同类工具的差异

维度	dsh	Claude Code / Codex	OpenCode	自建框架 (LangGraph 等)
定位	Agent 运行时 + 生态	闭源/半开源的 Agent 产品	开源 Agent 客户端	开发者自己拼
可定制性	一切皆插件，官方鼓励改	有限（配置/钩子）	中等（配置为主）	完全自由但费时
官方开源	✅ MIT	❌（闭源或受限）	✅	—
开箱即用	web + headless + 插件市场雏形	✅ 成熟	✅	需自建
生态现状	零日起步（2026-08-13 开源）	成熟	成熟	—
适合谁	想深度定制 + 玩生态的开发者	开箱即用	熟悉 OpenCode 的用户	需要完全控制的团队

现实定位：dsh 目前在“开箱即用”上不如 Claude Code 成熟（迭代快、有破坏性变更、中文教程此前为零），但在***可控性 × 开源 × 官方背书***这个组合上是独特的——这是它值得早入场的核心原因，也是生态项目的机会窗口。

1.5 生态现状（2026-08-13 快照）

- **官方：**仓库 12k+ commits、`dsh-plugin` topic 已建立、Discord 社区、CONTRIBUTING 鼓励生态。
- **社区：**插件生态刚萌芽——最完整的社区插件（DSH-better-sidebar：文件管理/终端/Git 面板）仅十位数 stars。
- **文档：**官方 README/架构文档完整，但**系统性新手教程为零**（本白皮书填补此空白）。

1.6 什么时候用它（选型建议）

推荐入场的场景：

- 你想要一个**模型无关、界面可选、行为可改**的 Agent 底座
- 你想开发 dsh 插件并成为生态早期贡献者（先发优势）
- 你需要在服务器/CI 跑 Agent（headless profile）

暂时观望的场景：

- 只要“开箱即用的编码助手”——Claude Code 等更成熟
- 不能接受快速迭代期的破坏性变更——生产核心依赖建议等 `0.1.0` 正式版

下一章：[第 2 章：五分钟快速上手](#) —— 装起来，跑起来。

第 2 章：五分钟快速上手

本章目标：装起来，跑起来——用最少的命令体验 dsh 的两种模式，并理解模型与推理档位。

2.1 安装

要求：Node.js ≥ 22（推荐 24）。一条命令：

```
# 直接运行 (npx 会自动下载)
npx -y @deepseek-ai/dsh --version

# 或全局安装
npm install -g @deepseek-ai/dsh
dsh --version
```

⚠️ **版本注意:** 当前为 `0.1.0-rc.x` (开发预览), 官方明示"将有破坏性变更"。教程示例基于 `0.1.0-rc.6` 验证。

2.2 模式一: Web UI (`dsh web`)

```
dsh web
# 输出: dsh web: http://127.0.0.1:3080
```

浏览器打开 <http://127.0.0.1:3080>:

- 左侧: 会话列表 / 工作区
- 中间: 对话区 (输入框、模型选择、推理等级)
- 右侧/底部: **由插件提供**的侧边栏 (默认空, 安装社区插件后出现)

首次使用会引导你配置模型 (DeepSeek API Key) 。

模型选择: 默认 `deepseek-v4-flash` (性价比) 或 `deepseek-v4-pro` (旗舰)。两者均支持**思考模式三档**:

推理等级	速度	质量	适用
low	最快	够用 (简单/确定性任务)	简单问答、批量、工具链中的廉价轮次
high (默认)	中等	好	日常 Agent 任务
max	最慢	最强	复杂推理、长链规划

💡 **性能提示:** 实测中, 模型在**每次工具调用前都会重新思考**——一个 50 步的工具链任务, "思考时间"可能占 90% 以上墙钟时间。 `low` 档对简单工具轮次几乎无损, 但能显著提速 (见第 6 章性能调优与社区插件 `dsh-tool-turbo`) 。

2.3 模式二: Headless (一次性任务, 适合脚本/CI)

```
dsh --profile headless "你好, 请用一句话介绍你自己"
# 打印最终回复后退出
```

Headless 的核心价值:

- **无 GUI 的自动化:** 可在 CI、服务器、cron 中使用
- **会话持久化:** 每次调用是"一个新鲜会话" (`--resume` 可恢复, 见 `dsh --profile headless --help`)
- **可脚本化:** 退出码非零即失败, 适合管道

2.4 配置文件在哪里

首次运行后生成:

```
~/ .dsh/
├── settings.yaml          # 全局设置（含默认模型与推理档位）
├── profiles/             # profile 目录（web / headless / 自定义）
│   └── web/
│       ├── package.json  # 插件依赖 + profile 清单
│       └── cordis.patch.yml # 你的补丁层（挂载插件）
├── sessions/            # 会话数据
└── storages/            # 持久化存储（workspace、session 索引等）
```

settings.yaml 示例：

```
agent-default-model:
  model: deepseek-v4-flash
  reasoningEffort: high
```

2.5 命令速查

命令	用途
dsh web	启动 Web UI（别名 dsh --profile web）
dsh --profile headless “任务”	一次性任务，打印结果后退出
dsh plugin --profile <name> add <pkg>	给 profile 安装插件
dsh --dump-config	打印当前 profile 的合成配置树
dsh --profile tui	TUI 模式（需先安装 tui 插件，官方未内置）

2.6 排障速查

现象	原因与解法
dsh: profile “tui” does not exist	tui profile 需插件创建（dsh plugin --profile tui add <pkg>），官方未内置
npx @deepseek-ai/dsh 极慢	首次下载包体较大（含 40+ 插件模块）；npm i -g 后更快
浏览器打不开 3080	检查端口占用：`netstat -ano`
模型无响应	检查 ~/.dsh/settings.yaml 的模型配置 + API Key
升级后行为变了	rc 阶段破坏性变更正常；留意官方 changelog

下一章：[第 3 章：profile 与插件系统](#)（规划中）—— 理解 dsh 的可定制骨架。

第 3 章：profile 与插件系统

本章目标：理解 dsh 的可定制骨架——profile 怎么组织、插件怎么挂载、host/client 双半是什么。**这是从“用户”变成“开发者”的分水岭。**

3.1 profile：一个可启动的配置栈

dsh 用 **profile** 表示"一种可启动的形态"。官方内置两个，其余用插件创建：

profile	用途	命令
web	Web UI（对话 + 侧边栏 + 工具）	dsh web
headless	一次性 CLI 任务	dsh --profile headless "任务"
tui（需插件）	终端 UI	dsh --profile tui（未内置，需安装插件）

一个 profile 目录长这样（`~/dsh/profiles/<name>/`）：

```
profiles/web/
├── package.json      # 插件依赖 + dsh.profile 清单（bundles 顺序）
├── cordis.patch.yml  # 你的补丁层：挂载/覆盖插件的声明
├── cordis.yml        # （生成的）合成配置
├── pnpm-workspace.yaml
└── node_modules/
```

加载顺序（官方文档原文）：内置 bundle（`dsh-base` → `dsh-web-app`）→ profile 的 `cordis.patch.yml` → 用户级 `~/dsh/cordis.patch.yml` → `--patch` 覆盖层。

3.2 挂载一个插件：两处改动

以挂载社区插件 `dsh-better-sidebar` 为例（真实操作）：

① `package.json` 加依赖（`link:` 指向本地源码，或用 `npm` 包名）：

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② `cordis.patch.yml` 加挂载行：

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 安装并重启：

```
cd ~/dsh/profiles/web && pnpm install
dsh web    # 重启后生效
```

💡 插件默认是**按会话隔离**的（`better-sidebar` 的布局/标签按会话持久化）——挂载是 **profile** 级，但状态是会话级。

3.3 host 半与 client 半：一个包，两副面孔

插件可以同时携带两个运行半：

半边	运行位置	职责	示例
host 半	Node 进程	工具、服务、事件、文件系统、进程	apply(ctx) 注册工具/服务
client 半	浏览器 (web profile)	UI、交互、DOM	package.json 的 dsh.client 声明 + src/client/

package.json 里如何声明 client 半：

```
{
  "dsh": {
    "client": {
      "inject": ["@deepseek-ai/dsh-client-runtime", "@deepseek-ai/dsh-client-locale"],
      "platform": "web"
    }
  },
  "exports": {
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
    "./client": { "types": "./lib/types/client/index.d.ts", "default": "./lib/client.js" }
  }
}
```

- host 半： exports["."] → apply(ctx) (cordis 插件主体)
- client 半： exports["./client"] → 浏览器侧的 apply(ctx)
- inject： 声明需要的服务 (cordis 依赖注入)

3.4 扩展点：改行为优先找钩子，别 fork 核心

官方原则 (CONTRIBUTING/AGENTS.md 明示)：***Plugins, not loop changes: new behavior goes on documented extension points***。新手最常见的错误是改核心 loop——正确做法是用扩展点。

已确认的常用扩展点（后续章节逐一实战）：

扩展点	位置	用途
agent/request waterfall	agent-loop	每次模型请求前改配置 (provider/model/reasoningEffort/tools) ——tool-turbo 用这个
agent/request-error	agent-loop	请求失败时干预 (官方 compaction 插件用这个做上下文溢出恢复)
conversationEvents.register	client runtime	订阅/注入对话事件 (tool/call、turn/start 等)
ctx.slots.inject	client ui-slots	在界面槽位注入 UI (如 turnTail 显示产物文件行)

settings 服务	dsh- settings	注册用户可配置的命名空间（设置页自动渲染）
ctx.provide / ctx.get	cordis	跨插件提供服务

3.5 常见坑（真实踩过）

坑	现象	解决
rc.1 依赖断裂	pnpm install 报 @deepseek-ai/dsh-type-meta@0.0.1-rc.1 404	官方 rc.1 时代多个包从未发布；升级到 ^0.1.0-rc.6 线
插件缺 main	dsh: No "exports" main defined	host 插件 package.json 要暴露入口 ("main": "src/index.ts" 可被 tsx 直接加载)
事件 handler 忘了 await next()	请求丢失 provider/model 报错	agent/request 的 next() 返回 Promise，必须 await 后 spread
类型报 'agent/request' is not assignable to keyof Events	npm 类型未 re-export 官方类型增强	用宽松签名 (ctx.on as unknown as ...) 在边界转换
client 包产物依赖 window.__ModuleLoader__	jsdom 无法直接跑 client 测试	组件级测试需 dsh web 运行时（官方 CI 跑）

下一章：第 4 章：插件开发实战（规划中）—— 从零写第一个 host 插件。

第 4 章：插件开发实战

本章目标：从零写一个真实可用的 host 插件——通过 agent/request 扩展点自动调节推理档位。这是社区项目 dsh-tool-turbo 的完整拆解，所有代码可运行、可测试。

4.1 我们要做什么

问题：dsh 在每次工具调用前模型都会重新思考（reasoning_effort）。一个 50 步工具链任务，"思考"占 90%+ 墙钟时间。

方案：一个 host 插件，监听 agent/request waterfall，根据当前步骤最近的工具调用，把简单轮次的 reasoning_effort 从 high 降到 low。

4.2 项目骨架

```
dsh-tool-turbo/
├── package.json      # host 插件声明
├── tsconfig.json
├── src/
│   ├── effort-decision.ts # 纯函数：决策逻辑（零依赖，可单测）
│   └── index.ts          # apply(ctx)：接入扩展点
```



```
└── tests/
    └── effort-decision.spec.ts
```

package.json 关键字段：

```
{
  "name": "dsh-tool-turbo",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

⚠️ 依赖版本务必用 `~0.1.0-rc.6` 线——`rc.1` 线的 `npm` 依赖链是断的（见第 3 章常见坑）。

4.3 纯函数：决策逻辑（零依赖，可单测）

src/effort-decision.ts：

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
  name: string // 工具名，如 'write'、'read'、'bash'
  argsSize: number // 参数大小（字符数）
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId // 用户基线档
  allowDowngrade: boolean
  allowUpgrade: boolean
}

const SIMPLE_TOOL_RE =
  /^(fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected // 全新提示：保持基线

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)
```

```

    if (ratio >= 0.75 && allowDowngrade) return 'low'
    if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
    if (ratio < 0.75) return allowUpgrade ? 'high' : selected
    return selected
  }

```

为什么拆成纯函数：决策逻辑与 dsh 运行时解耦——单元测试零依赖、毫秒级、覆盖所有分支，实机只需要验证“注入是否真的发生”。

4.4 插件主体：接入 agent/request waterfall

src/index.ts :

```

import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface ToolTurboConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: ToolTurboConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {
  const events = (agent as { session?: { events?: readonly unknown[] } }).session?.events ?? []
  const out: ToolCallSample[] = []
  for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
    const e = events[i] as { type?: string; data?: { name?: string; arguments?: unknown } } |
    undefined
    if (e?.type !== 'tool/call') continue
    out.push({
      name: e.data?.name ?? 'tool',
      argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
    })
  }
  return out.reverse()
}

export function apply(ctx: Context, config: ToolTurboConfig = DEFAULT_CONFIG): void {
  if (!config.enabled) return

  // 边界适配: npm 包未 re-export 官方事件类型增强, 这里放宽签名 (第 3 章常见坑)
  const on = ctx.on as unknown as (

```

```

    event: string,
    handler: (payload: Record<string, unknown>, next: () => unknown) => unknown | Promise<unknown>,
  ) => void

on('agent/request', async (payload, next) => {
  const seed = await next() as { reasoningEffort?: unknown } // ⚠️ 必须 await!
  const calls = recentToolCalls(payload.agent)
  const effort = decideEffort({
    recentCalls: calls,
    selected: config.baseline,
    allowDowngrade: config.allowDowngrade,
    allowUpgrade: config.allowUpgrade,
  })
  console.log(`[tool-turbo] calls=${JSON.stringify(calls)} => reasoningEffort=${effort}`)
  return { ...seed, reasoningEffort: effort }
})
}

```

三个关键点（都是真实踩过的坑）：

1. `next()` 是 **Promise**： `await next()` 拿到当前配置；不 `await` 直接 `spread` 会得到空对象 → `provider/model` 丢失 → 报错。
2. **waterfall** 语义：监听者的返回值传给下一个监听者/最终请求。返回 `{...seed, reasoningEffort}` 就是“保留原配置 + 覆盖推理档位”。
3. `agent/request` 每步都触发： `agent-loop` 的 `buildRequest` 在每一步都会走这个 waterfall——所以动态决策天然按步生效。

4.5 测试

单元测试（纯函数，零依赖）：

```

import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {
  expect(decideEffort({
    recentCalls: [{ name: 'write', argsSize: 40 }],
    selected: 'high', allowDowngrade: true, allowUpgrade: true,
  })).toBe('low')
})
// ... 更多分支：全新提示保持基线 / 禁用降档 / 超大载荷升 max / 混合工具升 high

```

实机验证（关键——证明“注入真的发生”）：

挂载插件（第 3 章方法）→ 重启 `dsh web` → 发一个创建文件的任务 → 观察 `dsh` 进程日志：

```

[tool-turbo] agent/request: calls=[] => reasoningEffort=high
[tool-turbo] agent/request: calls=[{"name":"write",...}] => reasoningEffort=low

```

第一轮无工具调用 → 保持基线 `high`；检测到 `write` 工具 → 下一轮降为 `low`。注入链路完整工作。

完整可运行代码：<https://github.com/Electricitysheep/dsh-tool-turbo>

4.6 给新手的三条开发纪律

1. **先找扩展点**：要改的行为 90% 有官方钩子（`agent/request`、`settings`、`conversationEvents`、`slots`）——不要 fork 核心。
2. **逻辑抽纯函数**：决策/计算逻辑与 dsh 解耦 → 单测毫秒级、覆盖全分支；实机只需验证“注入发生”。
3. **实机验证不能省**：单测证明逻辑，实机日志证明接线——两个都过才算完成。

下一章：第 5 章：实战案例（规划中）—— Git 面板、HTML 草稿预览、提速插件。

第 5 章：实战案例

本章目标：用三个**真实提交到开源仓库**的案例，演示插件的完整开发闭环——需求 → 实现 → 测试 → 实机验证。全部是社区真实 PR，可对照源码学习。

5.1 案例一：Git 面板补全 push / pull / fetch

背景：社区插件 `DSH-better-sidebar` 提供 Git 面板，但只有本地操作（暂存/提交/还原），**没有远端同步**。

实现（4 个文件，全部走“纯函数 + 路由 + UI”分层）：

```
src/git.ts           # 纯函数层：upstreamInfo / fetchRemote / pull / push
src/index.ts         # 路由层：git.upstream / git.fetch / git.pull / git.push
src/client/GitView.tsx # UI 层：上游徽标（↓ behind ↑ ahead）+ 三个按钮
tests/git-sync.spec.ts # 集成测试：本地 bare-repo 全链路
```

关键设计（值得抄的部分）：

```
// push 只允许 --force-with-lease，绝不裸 --force —— 安全红线文档化
export async function push(cwd: string, force = false): Promise<string> {
  const args = ['push']
  if (force) args.push('--force-with-lease')
  return runGit(cwd, args)
}
```

测试（本地 bare repo，无网络、无全局 git 配置）：

```
// 覆盖：上游跟踪 / 推送 / 拉取快进 / fetch 不动 HEAD / force-with-lease 拒绝覆盖他人提交
it('force push refuses to clobber unseen remote commits', async () => {
  // 其他 clone 先推了提交；本地改写历史但不 fetch (lease 过期)
  await expect(git.push(clone, true)).rejects.toThrow()
  // 远端仍持有对方提交 —— 安全保证生效
})
```

实机验证（Playwright 操作真实 dsh web）：

- 三按钮渲染，pull/push 在无 upstream 时正确禁用
- 点击“拉取远端”→ 网络请求 `git.fetch [200]` → 面板自动刷新

PR 见：<https://github.com/omdsh-dev/DSH-better-sidebar/pull/10>

5.2 案例二：HTML 预览支持未保存草稿

背景：HTML 文件编辑后，预览只显示已保存版本（README 已知限制）。看似简单，但有安全约束：非沙箱的 `srcdoc` `iframe` 会继承父 `origin`（官方代码注释明确）。

解决：抽纯函数做"安全决策"：

```
// 沙箱开启时 dirty 草稿才用 srcdoc (opaque origin, 安全)；
// 沙箱关闭时拒绝 srcdoc，保持 route-src (跨源保证)
export function htmlPreviewTarget(input): HtmlPreviewTarget {
  if (input.isHtml && input.dirty && input.draft !== null && !input.sandboxOff) {
    return { srcDoc: input.draft }
  }
  return { src: input.routeUrl }
}
```

教训：UI 看似"加个功能"，但安全模型是硬约束——先读代码注释里的为什么，再动手。

PR 见：<https://github.com/omdsh-dev/DSH-better-sidebar/pull/11>

5.3 案例三：tool-turbo 提速插件

(第 4 章完整拆解过，此处只讲结果)

- 决策逻辑：纯函数 `decideEffort`，6/6 单测
- 注入链路：`agent/request waterfall`，实机日志证明 `high` $\rightarrow$ `low`
- 价值定位（重要修正）：简单任务 `dsh` 本来就快；提速收益在长工具链任务——每步思考降档的累计效果

5.4 三个案例的共同方法论

- 纯函数隔离：决策/计算逻辑零依赖 $\rightarrow$ 单测全分支
- 扩展点接入：路由/`waterfall`/事件——不碰核心
- 实机验证闭环：单测（逻辑）+ 真实 `dsh` 日志/网络请求（接线）双证据

下一章：[第 6 章：进阶与性能调优](#)（规划中）。

第 6 章：进阶与性能调优

本章目标：从"能跑"到"跑得好"——推理档位策略、工具调用耗时分析、真实踩坑清单。

6.1 性能模型：dsh 的时间花在哪

实测一个"创建文件"任务的耗时分布：

阶段	占比	说明
模型思考 (Think)	~90%	每次工具调用前都会重新思考，是绝对大头
工具执行	<1%	文件写入等毫秒级
网络/渲染	~10%	API 往返 + UI 更新

推论：

- 简单任务 → 优化思考时间（降推理档）
- 长工具链任务 → 每步都省思考时间，累计收益最大
- 工具本身慢（搜索/大文件）→ 优化工具实现，不是调档

6.2 reasoning_effort 策略（官方三档）

档位	场景建议
low	简单/确定性轮次：文件操作、批量、工具链中的廉价步
high	日常 Agent 任务（默认）
max	复杂推理、长链规划、debug

手动： UI 的"推理等级"选择器，或 `~/dsh/settings.yaml` 的 `reasoningEffort` 。 **自动：** 插件按工具轮次动态降档（`dsh-tool-turbo`，第 4 章）。

6.3 工具调用耗时可视化

dsh 的会话统计行（Web UI 底部）显示：`N 轮 • M 步 | LLM Xs • 工具调用 Ys | 首 token 平均 ...` ——这是最快的瓶颈定位手段。

进阶：host 插件监听工具事件做 per-tool 计时（`tool-turbo` 已内置 host 日志版本），把"哪个工具最慢"暴露出来。

6.4 常见坑清单（真实踩过，含解法）

#	坑	现象	解法
1	rc.1 依赖断裂	<code>pnpm install</code> 404 (<code>dsh-type-meta</code> 等从未发布)	依赖用 <code>^0.1.0-rc.6</code> 线
2	插件缺 main	<code>No "exports" main defined</code>	暴露 . 入口; <code>"main": "src/index.ts"</code> 可被 <code>tsx</code> 加载
3	<code>next()</code> 忘 <code>await</code>	<code>provider/model</code> 丢失报错	<code>agent/request</code> 的 <code>next()</code> 返回 <code>Promise</code> ，必须 <code>await</code>
4	事件类型不识别	<code>'agent/request' is not assignable to keyof Events</code>	<code>npm</code> 未 <code>re-export</code> 类型增强，边界放宽签名
5	client 测试跑不了	<code>jsdom</code> 报 <code>window.__ModuleLoader__ undefined</code>	client 产物依赖 dsh 引导机制，组件测试在官方 CI 跑
6	简单任务"突然变快"的误判	<code>1s vs 110s</code> 差异被误归因	<code>DeepSeek context cache</code> 命中也会提速——A/B 测试要用全新 <code>prompt</code>
7	端口占用	<code>dsh web</code> 起不来	<code>`netstat -ano</code>

6.5 评测视角：官方成绩单 vs 独立实测

（结合 0813 正式版发布）看 Agent 模型成绩单要三问：

1. 谁测的？官方自测（自家 Harness）vs 独立第三方（AA 等）
2. 什么 harness？框架不同分数差很大（官方 `Terminal-Bench` 87.9 vs AA 独立 79）

3. 验证器多严？宽松验证器（SWE-bench Verified 8.5% 假阳性）vs 严格（DeepSWE 0.3%）

dsh 的 `agent/request` waterfall 让模型跑分可复现——这是它相比闭源产品的工程优势。

下一章：第 7 章：生态与资源（规划中）。

第 7 章：生态与资源

本章目标：给你一份“加入 dsh 生态”的地图——官方入口、社区现状、以及参与方式。

7.1 官方入口

资源	地址	用途
官方仓库	https://github.com/deepseek-ai/deepseek-harness	源码、架构文档、issue
API 文档	https://api-docs.deepseek.com	模型、定价、API 指南
Discord	官方 README 内链接	社区讨论
Discussions	官方仓库 Discussions	提案/求助（官方当前建议的贡献入口）

注意：官方 CONTRIBUTING 明确“目前不接受外部 PR”（2026-08-13 时点）——但鼓励：

- 在 Discussions 提建议（官方会评估）
- 做 **dsh-plugin 生态项目**（官方点名认可的方式）
- 写教程/博客

7.2 插件生态（2026-08-13 快照）

项目	定位	状态
DSH-better-sidebar	文件管理/终端/Git/浏览器侧边栏	社区最完整插件
dsh-tool-turbo	工具调用提速（reasoning_effort 自动调节）	社区提速插件
dsh-handbook（本白皮书）	新手教程	生态文档

发现插件：GitHub 搜 `topic:dsh-plugin`。发布插件：给你的仓库加 `dsh-plugin` topic + npm 发布。

7.3 如何参与生态（新手路径）

- 用起来：`dsh web` + 装两个社区插件，跑通日常
- 小改进：给社区插件提 PR（读第 5 章的三个案例，那是完整的 PR 范式）
- 发插件：从第 4 章的最小 host 插件起步，挂 `dsh-plugin` topic
- 写内容：教程/测评/避坑文（官方鼓励），与本白皮书互相引用

7.4 推荐阅读路径

目标	路径
快速上手	第 2 章 → 装 better-sidebar → 日常用

开发插件	第 3 章 → 第 4 章 → 抄第 5 章案例
性能调优	第 6 章 → tool-turbo 源码
深度定制	官方 AGENTS.md（架构）→ docs/architecture.md → packages/ 源码

结语

dsh 是 2026-08-13 才开源的项目——**生态的每一天都是"早期"**。白皮书会随 dsh 演进持续更新。如果某章命令失效，大概率是 rc 版本迭代所致——以官方 changelog 为准。

祝你在这个全新的生态里，抢到自己的位置。🚀